

KONCNO POROČILO PROJEKTNE NALOGE

Projekt: Pametni paketnik

Predmet: Računalniški vid

Člani ekipe in vloge:

- **Član 1:** Iris Skledar Hudoklin – zajem, priprava in argumentacija podatkov
- **Član 2:** Nika Zajšek – model in algoritem računalniškega vida
- **Član 3:** Manja Zdouc – API, integracija sistema in priprava okolja

Povezava do Git repozitorija: <https://github.com/pametni-paketnik/paketnik.git>

1. CILJ PROJEKTA

Glavni cilj projektne naloge je bil razvoj sistema, ki s pomočjo metod računalniškega vida rešuje problem identifikacije osebe preko FaceID-ja. Sistem je zasnovan kot modularna rešitev, ki obsega vse od zajema in predobdelave slik, klasifikacije s pomočjo nevronske mreže, do končne aplikacijske integracije v obliki API vmesnika.

2. UPORABLJENI POSTOPKI RAČUNALNIŠKEGA VIDA

V projektu so bili uporabljeni naslednji ključni postopki in metode računalniškega vida:

Predobdelava slik: Vhodni podatki so bili normalizirani in standardizirani, da se zagotovi stabilnost učenja. Izvedeni so bili postopki spreminjanja velikosti slik (resizing) na dimenzije 224*224, pretvorba v ustrezne barvne prostore (RGB) in uporabljene tehnike naključnega izrezovanja (random cropping), zrcaljenja (horizontal flip) in blage rotacije.

Globoko učenje (Deep Learning): Za klasifikacijo je bila izbrana konvolucijska nevronska mreža ResNet18. Uporabili smo pristop prenosnega učenja (Transfer Learning) z vnaprej naučenimi utežmi, pri čemer smo zadnji polno povezani sloj (fully connected layer) prilagodili in na novo natrenirali za specifične razrede našega problema.

Aplikacijski vmesnik (API): Za povezavo natreniranega modela v delujočo celoto je bil razvit aplikacijski vmesnik (API). Njegova glavna naloga je omogočanje komunikacije med uporabnikom (odjemalcem) in modelom računalniškega vida.

3. IMPLEMENTACIJA IN ARHITEKTURA SISTEMA

ZAJEM IN PREDOBDELAVA PODATKOV

Prva članica projekta je bila odgovorna za pretvorbo surovih vhodnih podatkov v obliko, ki jo nevronska mreža lahko procesira. Ta faza poteka v naslednjih korakih:

- **Zajem in strukturiranje:** Slike so bile zbrane in organizirane v standardno mapo s podmapami, ločenimi glede na razrede. Podatki so bili razdeljeni na učno (train), validacijsko (val) in testno (test) množico.
- **Geometrijske transformacije:** Ker model ResNet18 zahteva fiksno velikost vhoda, so vse slike interpolirane in spremenjene na dimenzijo 224*224 slikovnih pik.
- **Normalizacija**
- **Augmentacija:** Prted procesom učenja se nad slikami dinamično izvedejo operacije, kot so naključni vodoravni zrcalni obrati (horizontal flip), blage rotacije in prilagoditve svetilnosti.

RESNET18 KLASIFIKACIJSKI MODEL

Jedro računalniškega vida predstavlja konvolucijska nevronska mreža z arhitekturo ResNet18. Izbira te mreže temelji na njeni sposobnosti globokega učenja brez pojava izginjajočega gradienta.

- **Prenosno učenje (Transfer Learning):** Namesto učenja mreže od začetka, smo uporabili vnaprej naučene uteži (pretrained weights) na zbirki ImageNet. Prvotni konvolucijski sloji delujejo kot splošni ekstraktorji značilnosti (zaznavanje robov, tekstur in preprostih oblik).
- **Prilagoditev klasifikatorja:** Zadnji polno povezani sloj (Fully Connected Layer / Linear), ki je bil izvirno namenjen klasifikaciji v 1000 ImageNet razredov, je bil odstranjen. Zamenjan je bil z novim linearnim slojem, prilagojenim poljubnemu številu razredov.

API IN INTEGRACIJSKI SLOJ

Zadnja komponenta deluje kot komunikacijski most med modelom globokega učenja in končnimi uporabniki oziroma aplikacijami. Implementirana je v obliki REST API-ja s pomočjo ogrodja FastAPI.

- **Sprejem zahteve (Endpoint):** API izpostavlja vstopno točko POST /predict. Ta točka sprejme binarno datoteko (sliko) preko standardnega obrazca (multipart/form-data).
- **Sinhronizacija procesa:** Ko API prejme sliko, sproži funkcijo za predobdelavo, jo pošlje v model na inferenco in rezultat zapakira v format JSON. API poskrbi, da model teče v načinu za vrednotenje, kar izklopi računanje gradientov in optimizira hitrost odziva strežnika.

4. USPEŠNOST REŠITVE

Model ResNet18 je bil izbran zaradi odličnega razmerja med računsko zahtevnostjo in natančnostjo. Med učenjem smo optimizirali hiperparametre, kot sta stopnja učenja (learning rate), velikost paketa (batch size) in optimizator (SGD, Adam).

Uspešnost sistema smo ovrednotili na testni množici kot: validacijska natančnost/ izguba, učna natančnost/ izguba

Zaradi implementirane augmentacije podatkov je sistem izjemno robusten in ohranja visoko natančnost tudi pri slabših svetlobnih pogojih ali spremenjenem kotu zajema slik.

5. RAZDELITEV DELA IN VERZIONIRANJE KODE

Za verzioniranje kode in sodelovanje smo uporabljali sistem Git. Celotna zgodovina repozitorija in prispevki so transparentno vidni iz dnevnikov (commit logs). Delo je bilo razdeljeno skladno z zahtevami:

Članica 1 (Iris Skledar Hudoklin): Zajem in priprava podatkov

- Izdelala skripte za zajem in organizacijo učnih ter testnih podatkov v smiselno strukturo map.
- Implementirala postopke predobdelave (sprememba velikosti, normalizacija pik).
- Zasnovala in kodirala lastne postopke augmentacije podatkov za izboljšanje robustnosti.

Članica 2 (Nika Zajšek): Model računalniškega vida

- Izbrala in pripravila arhitekturo ResNet18 ter implementirala prenosno učenje.
- Izvedla učenje modela ter postopek optimizacije hiperparametrov.
- Ovrednotila uspešnost rešitve s testnimi metrikami in pripravila končni naučeni model za integracijo.

Članica 3 (Manja Zdouc): API in priprava okolja

- Razvila aplikacijski vmesnik (API) za povezavo modela z zunanjim svetom.
- Pripravila celotno okolje in zapakirala strežniški del z modelom v Docker vsebnik.
- Napisala zagonske skripte in pripravila tehnična navodila za uporabo sistema.